University Name : _____

College Name : _____

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

WEB TECHNOLOGY LABORATORY FILE

Name : _____

Roll Number : _____

Enrollment No. : _____

Course : B.Tech CSE

Semester : _____

Academic Session : _____

Submitted To : _____
(Faculty Name)

INDEX

S.No.	Name of Program	Date	Page No.
1	HTML program to create resume preparation using tables.		3
2	HTML program for home page creation using frames.		5
3	HTML program for form creation.		8
4	Create a web page to embed an image map in a web page using HTML.		10
5	Create a web page to fix the hot spots and to show all the related information when the hot spots are clicked using HTML.		12
6	Create a web page to get the coordinates from an image using JavaScript.		14
7	Create a web page with all types of cascading style sheets.		16
8	Write HTML/JavaScript to display your CV in navigator, your institute website, Department website and tutorial website for specific subject.		18
9	Design HTML form for keeping student record and validate it using JavaScript.		20
10	Writing program in XML for creation of DTD, which specifies set of rules.		23
11	Create a style sheet in CSS/XSL and display the document in Internet Explorer.		25

Practical No. 1

Program: HTML program to create resume preparation using tables.

Source Code:

File: resume.html

```
<!DOCTYPE html>
<html>
<head>
<title>Resume - Table Layout</title>
<style>
body { font-family: "Times New Roman", serif; }
table.outer { border-collapse: collapse; width: 100%; }
table.outer td { border: 1px solid black; padding: 6px; }
table.data { border-collapse: collapse; width: 100%; }
table.data th, table.data td {
border: 1px solid black; padding: 4px; text-align: left;
}
h2 { margin: 8px 0 4px 0; font-size: 16px; }
</style>
</head>
<body>

<!-- Resume built entirely with tables -->
<table class="outer">
<tr>
<td style="width:130px; text-align:center;">
<img src="photo.jpg" alt="Photo" width="110" height="130">
</td>
<td>
<h1 style="margin:0;">Ananya Sharma</h1>
<p style="margin:4px 0;">
B.Tech Computer Science &amp;amp; Engineering<br>
Address: _____, Lucknow, Uttar Pradesh<br>
Phone: ********** &amp;nbsp; Email: _____@mail.com
</p>
</td>
</tr>
</table>

<h2>Career Objective</h2>
<p>To work in a challenging environment where I can apply my
knowledge of web technologies and contribute to the growth
of the organization.</p>

<h2>Educational Qualification</h2>
<table class="data">
<tr><th>Qualification</th><th>Board/University</th>
<th>Year</th><th>Percentage</th></tr>
<tr><td>B.Tech CSE</td><td>_____ University</td>
<td>20XX</td><td>__%</td></tr>
<tr><td>Class XII</td><td>CBSE</td><td>20XX</td><td>__%</td></tr>
<tr><td>Class X</td><td>CBSE</td><td>20XX</td><td>__%</td></tr>
</table>
```

```
<h2>Technical Skills</h2>
<table class="data">
<tr><th>Area</th><th>Skills</th></tr>
<tr><td>Languages</td><td>C, C++, Java, Python</td></tr>
<tr><td>Web</td><td>HTML, CSS, JavaScript, XML</td></tr>
<tr><td>Database</td><td>MySQL</td></tr>
</table>

<h2>Declaration</h2>
<p>I hereby declare that the above information is true to the
best of my knowledge.</p>
<p style="text-align:right;">Signature: _____________</p>

</body>
</html>
```

Output/Result:

Representative browser output (open the program file in a browser to see the actual page):

Opening resume.html in a browser displays a one-page resume built entirely with HTML tables: a bordered top table holding the photo cell and the candidate's name, address, phone and email, followed by Career Objective, an Educational Qualification table, a Technical Skills table, and a Declaration with a signature line.

```
+-------+-------------------------------------+
| PHOTO | Ananya Sharma |
| | B.Tech CSE | Phone | Email |
+-------+-------------------------------------+
Career Objective
Educational Qualification
+---------------+-------------+------+-----+
| Qualification | Board/Univ. | Year | % |
+---------------+-------------+------+-----+
| B.Tech CSE | ... University| 20XX | .. |
+---------------+-------------+------+-----+
Technical Skills ... Declaration ... Signature
```

Practical No. 2

Program: HTML program for home page creation using frames.

Source Code:

File: index.html (frameset)

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Frameset//EN"
"http://www.w3.org/TR/html4/frameset.dtd">
<!-- Traditional HTML frames (HTML 4.01 Frameset). -->
<html>
<head>
<title>My Home Page - Frames</title>
</head>
<frameset rows="15%,*,12%">
<frame src="header.html" name="header" scrolling="no">
<frameset cols="25%,*">
<frame src="menu.html" name="menu">
<frame src="welcome.html" name="main">
</frameset>
<frame src="footer.html" name="footer" scrolling="no">
<noframes>
<body>Your browser does not support frames.</body>
</noframes>
</frameset>
</html>
```

File: header.html

```
<!DOCTYPE html>
<html>
<head><title>Header</title></head>
<body style="background:#1f3864; color:#ffffff; text-align:center;">
<h1 style="margin:12px;">MY PERSONAL HOME PAGE</h1>
</body>
</html>
```

File: menu.html

```
<!DOCTYPE html>
<html>
<head><title>Menu</title></head>
<body style="background:#d6e4f0;">
<h3>Menu</h3>
<ul>
<li><a href="welcome.html" target="main">Home</a></li>
<li><a href="about.html" target="main">About Me</a></li>
<li><a href="education.html" target="main">Education</a></li>
<li><a href="contact.html" target="main">Contact</a></li>
</ul>
</body>
</html>
```

File: welcome.html

```
<!DOCTYPE html>
<html>
<head><title>Welcome</title></head>
<body>
```

```
<h2>Welcome to my home page!</h2>
<p>This is the main content frame. Use the menu on the left
to navigate. Every link opens inside this frame because
of <b>target="main"</b>.</p>
</body>
</html>
```

File: about.html

```
<!DOCTYPE html>
<html>
<head><title>About</title></head>
<body>
<h2>About Me</h2>
<p>I am a B.Tech CSE student interested in web technologies.</p>
</body>
</html>
```

File: education.html

```
<!DOCTYPE html>
<html>
<head><title>Education</title></head>
<body>
<h2>Education</h2>
<ul>
<li>B.Tech CSE - _____ University</li>
<li>Class XII - CBSE</li>
<li>Class X - CBSE</li>
</ul>
</body>
</html>
```

File: contact.html

```
<!DOCTYPE html>
<html>
<head><title>Contact</title></head>
<body>
<h2>Contact</h2>
<p>Email: _____@mail.com<br>Phone: **********</p>
</body>
</html>
```

File: footer.html

```
<!DOCTYPE html>
<html>
<head><title>Footer</title></head>
<body style="background:#1f3864; color:#ffffff; text-align:center;">
<p style="margin:6px;">&copy; 2026 My Home Page</p>
</body>
</html>
```

Output/Result:

Representative browser output (open the program file in a browser to see the actual page):

Opening index.html divides the browser window into frames (traditional HTML 4.01 frameset): a dark-blue header (15%), a middle row with the menu (25%) and the main

content frame, and a footer (12%). Clicking a menu link loads that page inside the main frame because of target="main".

```
+-----------------------------------------+
| MY PERSONAL HOME PAGE (header) | 15%
+---------+-------------------------------+
| Menu | Welcome to my home page! |
| - Home | (main frame content) |
| - About | |
+---------+-------------------------------+
| (c) 2026 My Home Page (footer) | 12%
+-----------------------------------------+
```

Practical No. 3

Program: HTML program for form creation.

Source Code:

File: form.html

```
<!DOCTYPE html>
<html>
<head>
<title>Student Registration Form</title>
<style>
body { font-family: Arial, sans-serif; background: #eef3f8; }
form {
background: #ffffff; width: 420px; margin: 20px auto;
padding: 20px; border: 2px solid #1f3864; border-radius: 8px;
}
h2 { text-align: center; color: #1f3864; }
label { display: block; margin-top: 10px; font-weight: bold; }
input, select, textarea {
width: 100%; padding: 6px; margin-top: 4px;
border: 1px solid #999; border-radius: 4px;
}
.btns { text-align: center; margin-top: 14px; }
</style>
</head>
<body>
<form action="#" method="post">
<h2>Student Registration Form</h2>

<label>Full Name:</label>
<input type="text" name="name" placeholder="Enter full name">

<label>Password:</label>
<input type="password" name="password">

<label>Gender:</label>
<input type="radio" name="gender" value="Male"> Male
<input type="radio" name="gender" value="Female"> Female

<label>Hobbies:</label>
<input type="checkbox" name="hobby" value="Reading"> Reading
<input type="checkbox" name="hobby" value="Sports"> Sports
<input type="checkbox" name="hobby" value="Music"> Music

<label>Course:</label>
<select name="course">
<option>B.Tech CSE</option>
<option>B.Tech IT</option>
<option>B.Tech ECE</option>
</select>

<label>Address:</label>
<textarea name="address" rows="3"></textarea>

<label>Upload Photo:</label>
```

```
<input type="file" name="photo">

<div class="btns">
<input type="submit" value="Submit">
<input type="reset" value="Reset">
</div>
</form>
</body>
</html>
```

Output/Result:

Representative browser output (open the program file in a browser to see the actual page):

Opening form.html shows a centered, bordered Student Registration Form with text, password, radio, checkbox, dropdown, textarea and file fields, plus Submit and Reset buttons.

```
+-------------------------------------+
| Student Registration Form |
| Full Name: [.................] |
| Password: [.................] |
| Gender: (.) Male ( ) Female |
| Hobbies: [x] Reading [ ] Sports |
| Course: [ B.Tech CSE v ] |
| Address: [.....................] |
| [Submit] [Reset] |
+-------------------------------------+
```

Practical No. 4

Program: Create a web page to embed an image map in a web page using HTML.

Source Code:

File: image-map.html

```
<!DOCTYPE html>
<html>
<head>
<title>Image Map Example</title>
</head>
<body>
<h2>Campus Map - Image Map</h2>
<p>Click on any building on the map:</p>

<!-- Image with an embedded client-side image map -->
<img src="campus-map.jpg" alt="Campus map" width="600" height="400"
usemap="#campusmap">

<map name="campusmap">
<area shape="rect" coords="20,40,180,160"
href="#library" alt="Library" title="Library">
<area shape="rect" coords="200,40,380,160"
href="#labs" alt="Computer Labs" title="Computer Labs">
<area shape="rect" coords="400,40,580,160"
href="#canteen" alt="Canteen" title="Canteen">
<area shape="rect" coords="20,180,290,300"
href="#auditorium" alt="Auditorium" title="Auditorium">
<area shape="rect" coords="310,180,580,300"
href="#sports" alt="Sports Ground" title="Sports Ground">
<area shape="default" href="#top" alt="Campus">
</map>

<hr>
<h3 id="library">Library</h3>
<p>Central library with reading halls and issue counter.</p>
<h3 id="labs">Computer Labs</h3>
<p>Departmental labs with internet facility.</p>
<h3 id="canteen">Canteen</h3>
<p>Student canteen.</p>
<h3 id="auditorium">Auditorium</h3>
<p>Seminar and event hall.</p>
<h3 id="sports">Sports Ground</h3>
<p>Cricket and football ground.</p>
</body>
</html>
```

File: campus-map.jpg (image used by the map)

```
[binary image file - see the programs folder]
```

Output/Result:

Representative browser output (open the program file in a browser to see the actual page):

Opening image-map.html displays the campus map image with an embedded client-side image map (<img usemap> + <map>/<area>). The mouse pointer changes over each rectangular area, and clicking an area jumps to its description below the image.

```
Campus Map - Image Map
[ campus-map.jpg 600x400 with clickable blocks ]
Library | Computer Labs | Canteen
Auditorium | Sports Ground
---
Library: Central library with reading halls ...
```

Practical No. 5

Program: Create a web page to fix the hot spots and to show all the related information when the hot spots are clicked using HTML.

Source Code:

File: hotspots.html

```
<!DOCTYPE html>
<html>
<head>
<title>Hot Spots on an Image</title>
<script>
// Show the related information when a hot spot is clicked.
function showInfo(title, info) {
var box = document.getElementById("info");
box.innerHTML = "<b>" + title + "</b><br>" + info;
box.style.display = "block";
}
</script>
<style>
#info {
display: none; margin-top: 12px; padding: 10px; width: 580px;
border: 2px solid #1f3864; background: #eef3f8;
}
</style>
</head>
<body>
<h2>Campus Map - Hot Spots</h2>
<p>Click a hot spot on the map to see its details below:</p>

<img src="campus-map.jpg" alt="Campus map" width="600" height="400"
usemap="#hotspots">

<map name="hotspots">
<area shape="rect" coords="20,40,180,160" href="#"
onclick="showInfo('Library',
'Central library: 10,000+ books, reading halls, Wi-Fi.');
return false;">
<area shape="rect" coords="200,40,380,160" href="#"
onclick="showInfo('Computer Labs',
'Three labs with 120 systems for CSE students.');
return false;">
<area shape="rect" coords="400,40,580,160" href="#"
onclick="showInfo('Canteen',
'Canteen serves breakfast, lunch and snacks.');
return false;">
<area shape="rect" coords="20,180,290,300" href="#"
onclick="showInfo('Auditorium',
'500-seat auditorium for seminars and functions.');
return false;">
<area shape="rect" coords="310,180,580,300" href="#"
onclick="showInfo('Sports Ground',
'Ground for cricket, football and athletics.');
return false;">
```

```
</map>

<div id="info"></div>
</body>
</html>
```

File: campus-map.jpg (image with the hot spots)

```
[binary image file - see the programs folder]
```

Output/Result:

Representative browser output (open the program file in a browser to see the actual page):

Opening hotspots.html shows the same campus map with fixed hot spots. Clicking any hot spot runs the JavaScript showInfo() function, which displays that building's related information in the panel below the image - without reloading the page.

```
Campus Map - Hot Spots
[ campus map; click a building ]
+-----------------------------------------+
| Library |
| Central library: 10,000+ books, reading |
| halls, Wi-Fi. |
+-----------------------------------------+
```

Practical No. 6

Program: Create a web page to get the coordinates from an image using JavaScript.

Source Code:

File: coordinates.html

```
<!DOCTYPE html>
<html>
<head>
<title>Image Coordinates with JavaScript</title>
<script>
// Display the x, y coordinates of the clicked point on the image.
function getCoords(event) {
var x, y;
if (event.offsetX !== undefined) {
x = event.offsetX;
y = event.offsetY;
} else { // fallback for older browsers
var rect = event.target.getBoundingClientRect();
x = Math.round(event.clientX - rect.left);
y = Math.round(event.clientY - rect.top);
}
document.getElementById("x").value = x;
document.getElementById("y").value = y;
document.getElementById("msg").innerHTML =
"You clicked at coordinates (" + x + ", " + y + ")";
}
</script>
</head>
<body>
<h2>Click on the image to get its coordinates</h2>

<img src="lab-map.jpg" alt="Lab map" width="500" height="350"
onclick="getCoords(event)" style="cursor:crosshair;">

<p>
X coordinate: <input type="text" id="x" size="6" readonly>
Y coordinate: <input type="text" id="y" size="6" readonly>
</p>
<p id="msg" style="color:#1f3864; font-weight:bold;"></p>
</body>
</html>
```

File: lab-map.jpg (image clicked for coordinates)

```
[binary image file - see the programs folder]
```

Output/Result:

Representative browser output (open the program file in a browser to see the actual page):

Opening coordinates.html shows a gridded map. Clicking anywhere on the image calls getCoords(event); JavaScript reads offsetX/offsetY and fills the X and Y text boxes and prints the message below.

```
Click on the image to get its coordinates
[ gridded map 500x350 - click anywhere ]
X coordinate: [237] Y coordinate: [148]
You clicked at coordinates (237, 148)
```

Practical No. 7

Program: Create a web page with all types of cascading style sheets.

Source Code:

File: css-types.html

```
<!DOCTYPE html>
<html>
<head>
<title>Types of CSS</title>

<!-- (3) EXTERNAL CSS: rules kept in a separate .css file -->
<link rel="stylesheet" type="text/css" href="styles.css">

<!-- (2) INTERNAL / EMBEDDED CSS: rules inside the <head> -->
<style type="text/css">
h2.internal {
color: #ffffff;
background-color: #b03a2e;
padding: 8px;
}
p.internal {
color: #b03a2e;
font-style: italic;
border: 2px solid #b03a2e;
padding: 8px;
}
</style>
</head>
<body>
<h1 style="text-align:center;">Three Ways of Using CSS</h1>

<!-- (1) INLINE CSS: style attribute on the element itself -->
<h2 class="inline" style="color:#ffffff; background-color:#1f3864;
padding:8px;">
1. Inline CSS (style attribute)
</h2>
<p style="color:#1f3864; border:2px solid #1f3864; padding:8px;">
This paragraph is styled with an inline <b>style</b> attribute.
</p>

<!-- (2) INTERNAL CSS -->
<h2 class="internal">2. Internal CSS (&lt;style&gt; in head)</h2>
<p class="internal">
This paragraph is styled by the internal style sheet.
</p>

<!-- (3) EXTERNAL CSS -->
<h2 class="external">3. External CSS (styles.css file)</h2>
<p class="external">
This paragraph is styled by the external style sheet.
</p>
</body>
</html>
```

File: styles.css (external style sheet)

```
/* External style sheet (linked with <link> tag) */
body {
background-color: #f4f7fb;
font-family: Georgia, serif;
}
h2.external {
color: #ffffff;
background-color: #2e7d32;
padding: 8px;
}
p.external {
color: #2e7d32;
font-size: 16px;
border: 2px dashed #2e7d32;
padding: 8px;
}
```

Output/Result:

Representative browser output (open the program file in a browser to see the actual page):

Opening css-types.html shows the same heading/paragraph styled three times: blue (inline style attribute), red (internal <style> in the head) and green (external styles.css linked with <link>).

```
Three Ways of Using CSS
+----------------------------------------+
| 1. Inline CSS (style attribute) [blue] |
+----------------------------------------+
+----------------------------------------+
| 2. Internal CSS (<style> in head) [red] |
+----------------------------------------+
+----------------------------------------+
| 3. External CSS (styles.css) [green] |
+----------------------------------------+
```

Practical No. 8

Program: Write HTML/JavaScript to display your CV in navigator, your institute website, Department website and tutorial website for specific subject.

Source Code:

File: links.html

```
<!DOCTYPE html>
<html>
<head>
<title>My Links - CV, Institute, Department, Tutorial</title>
<script>
// Open each website in the browser (navigator) using JavaScript.
function openPage(url) {
window.open(url, "_blank");
}
function openCV() { openPage("cv.html"); }
function openInstitute() {
// Replace with your own institute website.
openPage("https://www.example.edu");
}
function openDepartment() {
// Replace with your own department website.
openPage("https://www.example.edu/cse");
}
function openTutorial() {
openPage("https://www.w3schools.com/html/");
}
</script>
</head>
<body style="font-family:Arial; text-align:center;">
<h2>Quick Links</h2>
<p>Click a button to open the page in the browser:</p>
<p><button onclick="openCV()">My CV</button></p>
<p><button onclick="openInstitute()">Institute Website</button></p>
<p><button onclick="openDepartment()">Department Website</button></p>
<p><button onclick="openTutorial()">
Tutorial Website (HTML - W3Schools)</button></p>
</body>
</html>
```

File: cv.html (CV page opened in the navigator)

```
<!DOCTYPE html>
<html>
<head><title>My CV</title></head>
<body style="font-family:Georgia, serif; margin:30px;">
<h1>Ananya Sharma</h1>
<p>B.Tech Computer Science &amp;amp; Engineering<br>
Email: _____@mail.com | Phone: **********</p>
<hr>
<h3>Profile</h3>
<p>Second year B.Tech CSE student learning web technologies.</p>
<h3>Skills</h3>
```

```
<ul>
<li>HTML, CSS, JavaScript</li>
<li>XML, XSL</li>
<li>C, Python</li>
</ul>
</body>
</html>
```

Output/Result:

Representative browser output (open the program file in a browser to see the actual page):

Opening links.html shows four buttons. Each button calls a JavaScript function that uses window.open() to display the page in the browser: the local CV page, the institute website, the department website, and an HTML tutorial site (replace the example URLs with your own).

```
Quick Links
Click a button to open the page in the browser:
[ My CV ] -> opens cv.html
[ Institute Website ] -> opens institute site
[ Department Website ] -> opens CSE department site
[ Tutorial Website (HTML - W3Schools) ]
```

Practical No. 9

Program: Design HTML form for keeping student record and validate it using JavaScript.

Source Code:

File: student-record.html

```
<!DOCTYPE html>
<html>
<head>
<title>Student Record Form with JavaScript Validation</title>
<style>
body { font-family: Arial, sans-serif; background: #eef3f8; }
form {
background: #fff; width: 430px; margin: 20px auto;
padding: 20px; border: 2px solid #1f3864; border-radius: 8px;
}
h2 { text-align: center; color: #1f3864; }
label { display: block; margin-top: 8px; font-weight: bold; }
input, select {
width: 100%; padding: 6px; margin-top: 3px;
border: 1px solid #999; border-radius: 4px;
}
.error { color: red; font-size: 12px; }
#result { text-align: center; font-weight: bold; color: green; }
</style>
<script>
function validateForm() {
var name = document.getElementById("name").value.trim();
var roll = document.getElementById("roll").value.trim();
var email = document.getElementById("email").value.trim();
var phone = document.getElementById("phone").value.trim();
var marks = document.getElementById("marks").value.trim();
var err = "";

// Name: letters and spaces only, at least 3 characters
if (!/^[A-Za-z ]{3,}$/.test(name))
err += "Name must contain letters only (min 3).\n";

// Roll number: letters and digits only
if (!/^[A-Za-z0-9]+$/.test(roll))
err += "Roll No. must be alphanumeric.\n";

// Email: standard pattern
var mailRe = /^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$/;
if (!mailRe.test(email))
err += "Enter a valid email address.\n";

// Phone: exactly 10 digits
if (!/^[0-9]{10}$/.test(phone))
err += "Phone must be exactly 10 digits.\n";

// Marks: number between 0 and 100
if (isNaN(marks) || marks < 0 || marks > 100)
```

```
err += "Marks must be a number between 0 and 100.\n";

var out = document.getElementById("result");
if (err !== "") {
out.style.color = "red";
out.innerHTML = "Errors:<br>" + err.replace(/\n/g, "<br>");
return false; // stop form submission
}
out.style.color = "green";
out.innerHTML = "Record saved successfully for " + name + "!";
return false; // demo: do not actually submit
}
</script>
</head>
<body>
<form onsubmit="return validateForm()">
<h2>Student Record</h2>

<label>Name:</label>
<input type="text" id="name">

<label>Roll No.:</label>
<input type="text" id="roll">

<label>Email:</label>
<input type="text" id="email">

<label>Phone:</label>
<input type="text" id="phone">

<label>Marks (0-100):</label>
<input type="text" id="marks">

<label>Course:</label>
<select id="course">
<option>B.Tech CSE</option>
<option>B.Tech IT</option>
</select>

<div style="text-align:center; margin-top:12px;">
<input type="submit" value="Save Record" style="width:auto;">
</div>
<p id="result"></p>
</form>
</body>
</html>
```

Output/Result:

Representative browser output (open the program file in a browser to see the actual page):

Opening student-record.html shows the Student Record form. On submit, JavaScript validateForm() checks every field (name letters-only, alphanumeric roll no., email pattern, 10-digit phone, marks 0-100). Invalid input shows the errors in red; valid input shows a success message.

```
Student Record
Name: [abc] Roll No.: [12!] Marks: [120]
Errors:
Name must contain letters only (min 3).
Roll No. must be alphanumeric.
Marks must be a number between 0 and 100.
--- valid input ---
Record saved successfully for Ananya Sharma!
```

Practical No. 10

Program: Writing program in XML for creation of DTD, which specifies set of rules.

Source Code:

File: students.xml (XML with internal DTD)

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- DTD written as an INTERNAL subset inside the XML file.
The DTD specifies the set of rules every <students> document
must follow. Open this file in a browser to view it. -->
<!DOCTYPE students [
<!ELEMENT students (student+)>
<!ELEMENT student (name, roll, course, marks)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT roll (#PCDATA)>
<!ELEMENT course (#PCDATA)>
<!ELEMENT marks (#PCDATA)>
<!ATTLIST student id CDATA #REQUIRED>
]>
<students>
<student id="S101">
<name>Ananya Sharma</name>
<roll>2301234</roll>
<course>B.Tech CSE</course>
<marks>88</marks>
</student>
<student id="S102">
<name>Rahul Verma</name>
<roll>2301235</roll>
<course>B.Tech CSE</course>
<marks>76</marks>
</student>
<student id="S103">
<name>Priya Singh</name>
<roll>2301236</roll>
<course>B.Tech CSE</course>
<marks>91</marks>
</student>
</students>
```

File: students-external.dtd (external DTD)

```
<!-- External DTD: the same rules kept in a separate .dtd file.
students-external.xml references it with:
<!DOCTYPE students SYSTEM "students-external.dtd"> -->

<!ELEMENT students (student+)>
<!ELEMENT student (name, roll, course, marks)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT roll (#PCDATA)>
<!ELEMENT course (#PCDATA)>
<!ELEMENT marks (#PCDATA)>
<!ATTLIST student id CDATA #REQUIRED>
```

File: students-external.xml (XML using external DTD)

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- XML document validated against the EXTERNAL DTD file -->
<!DOCTYPE students SYSTEM "students-external.dtd">
<students>
<student id="S101">
<name>Ananya Sharma</name>
<roll>2301234</roll>
<course>B.Tech CSE</course>
<marks>88</marks>
</student>
<student id="S102">
<name>Rahul Verma</name>
<roll>2301235</roll>
<course>B.Tech CSE</course>
<marks>76</marks>
</student>
</students>
```

Output/Result:

Representative browser output (open the program file in a browser to see the actual page):

Opening students.xml in a browser displays the XML document as a collapsible tree. The internal DTD subset declares the rules: students contains one or more student elements; each student has name, roll, course, marks and a required id attribute. students-external.xml follows the same rules kept in the external file students-external.dtd.

```
- <students>
- <student id="S101">
<name>Ananya Sharma</name>
<roll>2301234</roll>
<course>B.Tech CSE</course>
<marks>88</marks>
+ <student id="S102"> ...
+ <student id="S103"> ...
```

Practical No. 11

Program: Create a style sheet in CSS/XSL and display the document in Internet Explorer.

Source Code:

File: students.xml (with xml-stylesheet instruction)

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- The xml-stylesheet processing instruction tells the browser
to apply students.xsl and display the result as a web page. -->
<?xml-stylesheet type="text/xsl" href="students.xsl"?>
<students>
<student id="S101">
<name>Ananya Sharma</name>
<roll>2301234</roll>
<course>B.Tech CSE</course>
<marks>88</marks>
</student>
<student id="S102">
<name>Rahul Verma</name>
<roll>2301235</roll>
<course>B.Tech CSE</course>
<marks>76</marks>
</student>
<student id="S103">
<name>Priya Singh</name>
<roll>2301236</roll>
<course>B.Tech CSE</course>
<marks>91</marks>
</student>
</students>
```

File: students.xsl (XSL style sheet)

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- XSL style sheet: converts the XML data into an HTML table. -->
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

<xsl:template match="/">
<html>
<head>
<title>Student Records (XML + XSL)</title>
<style>
body { font-family: Arial, sans-serif; }
h2 { color: #1f3864; }
table { border-collapse: collapse; width: 70%; }
th, td { border: 1px solid #333; padding: 6px; }
th { background: #1f3864; color: #fff; }
tr:nth-child(even) { background: #eef3f8; }
</style>
</head>
<body>
<h2>Student Records</h2>
<table>
```

```
<tr>
<th>ID</th><th>Name</th><th>Roll No.</th>
<th>Course</th><th>Marks</th>
</tr>
<xsl:for-each select="students/student">
<tr>
<td><xsl:value-of select="@id"/></td>
<td><xsl:value-of select="name"/></td>
<td><xsl:value-of select="roll"/></td>
<td><xsl:value-of select="course"/></td>
<td><xsl:value-of select="marks"/></td>
</tr>
</xsl:for-each>
</table>
</body>
</html>
</xsl:template>

</xsl:stylesheet>
```

Output/Result:

Representative browser output (open the program file in a browser to see the actual page):

Opening students.xml in a browser applies students.xsl automatically (through the <?xml-stylesheet?> instruction) and renders a styled HTML table of the student records. Browser compatibility: XSLT 1.0 is supported by Internet Explorer 5.5+ and by all modern browsers (Chrome, Edge, Firefox), so the XML file can be opened directly - no web server is needed.

```
Student Records
+------+--------------+---------+------------+-------+
| ID | Name | Roll No.| Course | Marks |
+------+--------------+---------+------------+-------+
| S101 | Ananya Sharma | 2301234 | B.Tech CSE | 88 |
| S102 | Rahul Verma | 2301235 | B.Tech CSE | 76 |
| S103 | Priya Singh | 2301236 | B.Tech CSE | 91 |
+------+--------------+---------+------------+-------+
```